

# Foundational Paradigms in Agent-Oriented Systems Programming: Security, Compiler Architecture, and the Design of a High-Assurance Runtime

## 1. Introduction to the Agentic Shift in Software Engineering

The rapid evolution of artificial intelligence throughout late 2024 and 2025 has precipitated a fundamental paradigm shift in software engineering and system architecture. Prior to this inflection point, the state-of-the-art in natural language processing was dominated by single-pass, autoregressive predictors optimized primarily to continue text sequences.<sup>1</sup> These large language models (LLMs) functioned as conversational utilities or static code-generation assistants. However, the landscape has rapidly transitioned toward autonomous, multi-agent systems—intelligent entities capable of environmental perception, strategic self-planning, and autonomous action execution.<sup>2</sup> This evolution from static generation to dynamic, agentic workflows necessitates a fundamental reevaluation of the execution environments and programming languages utilized to facilitate machine-driven logic.

The integration of AI agents into production environments has democratized software creation, but it has simultaneously catalyzed what industry analysts describe as the "world's largest shadow IT problem".<sup>4</sup> The phenomenon of "vibe coding"—where end-users rapidly prototype applications using AI without rigorous architectural oversight—has resulted in the proliferation of unverified code, inadvertent exposure of credentials, and the deployment of applications lacking fundamental security audits.<sup>4</sup> Demos and prototypes are easily generated, but translating these agent-generated scripts into robust, production-ready software introduces severe friction, overloading traditional software development lifecycles with unpredictable, non-deterministic logic.<sup>4</sup>

At the core of this friction is a fundamental mismatch: autonomous agents are forced to operate within programming languages and execution runtimes designed exclusively for human cognitive models.<sup>5</sup> Traditional languages prioritize human-readable syntax, dynamic typing for developer velocity, and complex, human-oriented debugging mechanisms. Conversely, LLMs and autonomous agents struggle with implicit state, require structured, deterministic feedback loops, and introduce unprecedented system-level security vulnerabilities when granted execution privileges.<sup>4</sup>

To mitigate these systemic risks and unlock the full potential of agentic computing, the industry is witnessing the emergence of a new class of "agent-oriented programming languages" and

high-assurance runtime environments.<sup>3</sup> This comprehensive research report provides an exhaustive analysis of the state-of-the-art in agent-oriented programming, with a specialized focus on the architectural decisions underlying Vercel Labs' recently introduced "Zero" language.<sup>9</sup> Furthermore, it investigates the critical security surfaces exposed by LLM-driven execution, including prompt-injection-based remote code execution (RCE).<sup>10</sup> By evaluating the trade-offs between Just-In-Time (JIT) compilation frameworks—specifically LLVM and Cranelift<sup>12</sup>—and exploring capability-based security models<sup>14</sup>, this research culminates in a rigorous specification for a novel, high-assurance agent language and Rust-based compiler architecture. This proposed approach targets near-native performance via a JIT compiler optimized for ARM64, RISC-V, and AMD64 architectures<sup>16</sup>, establishing a secure, verifiable execution substrate for the next generation of autonomous AI systems.

## **2. The Evolution and Taxonomy of Agent-Oriented Programming**

The concept of Agent-Oriented Programming (AOP) is not novel; its theoretical foundations were established by researchers such as Yoav Shoham in 1994, who proposed treating agents as fundamental computational units characterized by reasoning, objectives, and interactions, analogous to how Object-Oriented Programming (OOP) treats objects.<sup>2</sup> However, the integration of these theoretical paradigms with the empirical capabilities of modern LLMs has necessitated the development of entirely new linguistic and architectural frameworks. The contemporary landscape of agent languages can be broadly categorized into distinct tiers, each addressing different layers of the abstraction stack.

### **2.1 Prompt-Level Control and Domain-Specific Languages**

At the foundational layer, specialized languages have emerged to constrain and control the stochastic nature of single prompt executions. LLMs are inherently non-deterministic due to token sampling, floating-point arithmetic variations across GPU clusters, Mixture-of-Experts routing, and batch-level dependencies.<sup>18</sup> This non-determinism presents a massive barrier to enterprise adoption, where auditability and reproducible behavior are strict requirements.<sup>18</sup> To impose structure on probability distributions, researchers have developed prompt-level domain-specific languages (DSLs) such as LMQL (Language Model Query Language) and Guidance.<sup>21</sup> LMQL, for instance, introduces the concept of Language Model Programming (LMP), utilizing a declarative, SQL-like syntax that combines natural language prompting with strict scripting instructions.<sup>22</sup> This approach allows developers to define multiple input and output variables within a single prompt, optimizing the overall statistical likelihood across numerous API calls and effectively managing complex interaction trees.<sup>22</sup>

Bridging the gap between front-end language design and backend runtime efficiency is SGLang, a framework uniquely positioned to enhance execution speed. Unlike its predecessors, which often prioritize syntax while overlooking hardware utilization, SGLang integrates Pythonic primitives with deep backend optimizations, such as RadixAttention,

enabling order-of-magnitude improvements in runtime efficiency and offering both interpreter and compiler execution modes.<sup>21</sup>

Furthermore, hyper-specialized DSLs have demonstrated significant efficacy in reducing agent hallucination within specific scientific and engineering domains. Frameworks like Anka are designed exclusively for data transformation pipelines, prioritizing syntax structures that prevent LLM coding errors rather than focusing on human developer ergonomics.<sup>24</sup> Similarly, the 3DGen framework utilizes a multi-agent architecture to translate informal RFC networking prose into a formal DSL, which is then compiled into provably correct, memory-safe C or Rust parsers.<sup>25</sup> These implementations prove that restricting the expressive freedom of an LLM through a highly constrained DSL drastically reduces non-deterministic failure modes.<sup>25</sup>

## **2.2 Orchestration-Level Frameworks**

Moving up the abstraction stack, orchestration frameworks such as LangChain, CrewAI, AutoGen, and AgentScope focus on multi-agent collaboration patterns.<sup>21</sup> These systems dictate how tools are registered, how agents exchange messages, and how state is preserved across long-running tasks. CrewAI introduced robust role-based multi-agent collaboration, while MetaGPT proposed structured coordination mediated by generated artifacts.<sup>28</sup>

Within specialized architectures, the concept of the Intelligent Specialized Agent (ISA) has gained traction. An ISA acts as a domain-specialized module designed to handle specific tasks, such as processing a particular data modality or executing a distinct analysis function, coordinating with supporting agents that manage external search and retrieval.<sup>29</sup> This modularity allows complex reasoning tasks to be subdivided, mapped, and executed in parallel or sequentially based on strict dependency rules.<sup>32</sup>

However, while orchestration frameworks successfully manage agent lifecycles and prompt pipelines, they fundamentally rely on the underlying host language (typically Python, JavaScript, or TypeScript) for the actual execution of the logic generated by the agents. This reliance exposes a profound structural weakness in the current agentic paradigm.

## **2.3 The Inherent Paradox of Human-Centric Languages in AI Code Generation**

When an AI agent is tasked with writing, executing, and debugging a software module, instructing it to use Python or JavaScript introduces significant friction. Programming languages have historically been designed to be fast, safe, and primarily human-friendly, prioritizing features like scope isolation, modularity, and syntactic sugar that aids human comprehension.<sup>5</sup>

However, LLMs do not require syntactic sugar; they require explicit structural boundaries. Human-oriented design often implements safeguards that become entirely redundant or actively harmful when the programmer is an AI. For example, implicit asynchronous behaviors, mandatory garbage collection (GC) that introduces non-deterministic execution pauses, and deeply nested object-oriented hierarchies force the LLM to maintain massive amounts of implicit context during code generation.<sup>6</sup>

More critically, traditional compilers and interpreters are engineered to emit unstructured, natural-language error messages designed for human eyes. When an agent compiles code and encounters a failure, it must parse a block of text, rely on its probabilistic reasoning engine to interpret the human-readable warning, and hallucinate a potential fix.<sup>9</sup> This reliance on non-deterministic interpretation of unstructured errors creates an inefficient, failure-prone debugging loop, fundamentally limiting the autonomy and reliability of the agent.<sup>33</sup>

## 3. Vercel Labs' "Zero": A Blueprint for Agent-Native Systems Languages

Recognizing the severe limitations of deploying agents within human-centric language environments, Vercel Labs introduced an experimental systems programming language explicitly designed for AI coding agents, codenamed "Zero".<sup>9</sup> Utilizing the highly minimalistic .O file extension, Zero shifts the target audience of the compiler from the human developer to the LLM itself, aiming to facilitate faster, smaller, and easier code generation and automated repair from "day zero".<sup>9</sup>

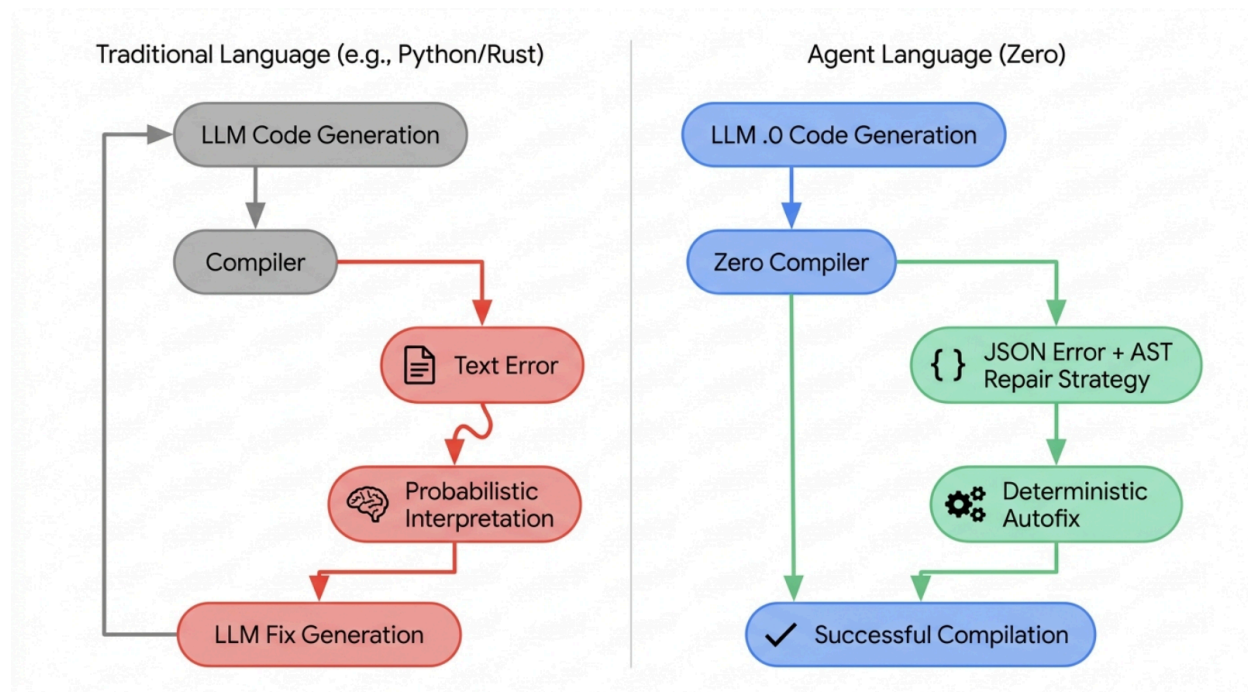
### 3.1 Core Design Paradigms of Zero

The architecture of Zero relies on several radical design departures from traditional languages, engineered to align directly with the cognitive constraints and strengths of language models.<sup>9</sup>

1. **JSON-Native Diagnostics and Deterministic Repair:** The most profound innovation of Zero is its compiler output. Instead of throwing human-readable text warnings, the Zero compiler emits highly structured, stable JSON payloads containing precise error codes, abstract syntax tree (AST) coordinates, and explicit repair metadata.<sup>9</sup> If a symbol is missing, the compiler does not simply state "Symbol not found"; it outputs a structured directive such as "declare-missing-symbol". This allows the AI agent to parse the JSON and deterministically patch the code using direct string manipulation or AST updates, completely bypassing the need for a secondary, error-prone probabilistic reasoning pass to interpret the error.<sup>9</sup>
2. **Explicit Effects and Eradication of Global State:** Zero enforces strict, explicit effect typing. Function signatures must exhaustively declare every resource they interact with. There are no hidden globals, no implicit async operations, and no hidden side effects.<sup>9</sup> For instance, a basic program requires the injection of a World object to interact with standard output: `pub fun main(world: World) -> Void raises { check world.out.write("hello from zero") }`.<sup>9</sup> This design allows an LLM to accurately predict the total scope of a function simply by analyzing its signature.
3. **Predictable Memory Management:** Zero opts for static dispatch and explicit allocation, deliberately avoiding the runtime tax and non-deterministic execution pauses associated with mandatory garbage collection.<sup>9</sup> This focus on predictable memory facilitates the creation of small, easily reasoned native tools.
4. **Structured Toolchain Interrogation:** The language ecosystem includes a suite of commands designed to emit deep architectural insights directly to agents via JSON.

Commands such as `zero graph --json` (dependency mapping), `zero size --json` (binary profiling), and `zero routes --json` provide high-fidelity, machine-readable feedback loops that allow an agent to understand the broader system architecture without parsing complex file systems.<sup>9</sup>

## Compiler Feedback Loops: Human-Centric vs. Agent-Centric Workflows



In traditional workflows, AI agents must probabilistically interpret human-readable compiler errors, leading to hallucinations and repeated failures. Vercel Zero's architecture emits structured JSON diagnostics containing exact AST coordinates and repair instructions, enabling deterministic code correction.

### 3.2 Compilation and Current Limitations

The execution model of Zero is tailored for native execution, allowing agents to build and compile source files into executables directly targeting platforms like `linux-musl-x64` using direct emitters, bypassing the need for an external C compiler in the end-user environment.<sup>34</sup> The repository features a native compiler implementation primarily authored in C (constituting 65.5% of the codebase), alongside compiler sources authored in Zero itself (`compiler-zero`) and testing frameworks utilizing JavaScript and Shell scripts.<sup>35</sup>

However, the architecture of Zero presents critical limitations. First, it is highly experimental; both the language specification and the compiler are actively changing.<sup>9</sup> More pressingly, the heavy reliance on C for the core compiler implementation introduces traditional memory safety

risks.<sup>35</sup> While Zero rigorously constrains the syntax the agent can write, it does not inherently solve the systemic challenge of executing potentially adversarial code within a zero-trust environment. A secure syntax is futile if the underlying compiler or execution runtime can be compromised.

## **4. The Cybersecurity Threat Landscape of LLM-Driven Runtimes**

The fundamental security paradox of agent-oriented programming is that the execution substrate must be highly capable to be useful—possessing access to file systems, network sockets, and execution contexts—yet it must be totally restricted to prevent catastrophic exploitation.<sup>36</sup> Large language models are highly susceptible to adversarial manipulation. Consequently, any agent language, standard library, and runtime environment must operate under a strict zero-trust assumption, treating the AI agent itself as a potentially malicious actor.<sup>11</sup>

### **4.1 Prompt Injections as Remote Code Execution (RCE)**

In a traditional chatbot context, a prompt injection merely manipulates the linguistic output of the model, resulting in reputational damage or misinformation. However, in an agentic framework equipped with tool-calling capabilities and a native compiler, a successful prompt injection translates directly into arbitrary Remote Code Execution (RCE).<sup>7</sup>

The attack chain is straightforward but devastatingly effective. An attacker embeds an injected prompt within an untrusted data source—such as a PDF document, an email, or a webpage that the agent is tasked with summarizing. When the LLM processes this context, the adversarial instruction overrides the system prompt, coercing the agent to utilize its execution tools to generate a malicious payload inside the container.<sup>7</sup> Because the agent writes the code itself, the generated script often appears legitimate and functional, evading traditional static security scanning approaches that look for known malware signatures.<sup>36</sup>

Once the payload is written and compiled, the agent executes it, leading to a sandbox escape, unauthorized data exfiltration, or complete host takeover.<sup>7</sup> Academic evaluations of 18 state-of-the-art LLMs functioning as reasoning engines within autonomous agents—including GPT-4, Claude 3.5, and Gemini-2.5—reveal an alarming vulnerability matrix.<sup>11</sup> Extensive testing demonstrates that 94.4% of these models succumb to Direct Prompt Injection designed to achieve computer takeover, and 83.3% fail against Retrieval-Augmented Generation (RAG) backdoors, autonomously installing and executing malware on victim machines without human intervention.<sup>11</sup>

### **4.2 The Insufficiency of Traditional Containerization**

To contain the blast radius of agent-generated logic, execution must be forcefully isolated. The industry standard approach of utilizing Docker containers is structurally insufficient for executing wholly untrusted, adversarial code generated by an AI. Standard containers rely on Software Fault Isolation (SFI) operating atop a shared host OS kernel; they utilize namespaces

and cgroups to partition resources.<sup>37</sup> However, as the documentation for hardened runtimes like gVisor explicitly states, "with standard containers, the workload is only one system call away from host compromise".<sup>38</sup> An agent that successfully generates a kernel exploit can shatter the container boundary and compromise the underlying infrastructure.

To achieve stringent isolation, platforms executing agent code must shift to hardware-level virtualization. Vercel Sandbox, for example, utilizes Firecracker MicroVMs to provide on-demand, isolated compute environments.<sup>37</sup> Firecracker provisions a dedicated, lightweight Linux kernel per sandbox, establishing a robust virtualization boundary that prevents container escapes while maintaining startup times measured in milliseconds.<sup>37</sup> These ephemeral, fast-booting environments are critical not only for security but also for testing the non-deterministic outputs of AI agents at scale, allowing for chaos engineering and probabilistic validation.<sup>40</sup>

| Aspect                 | Docker Containers                                                                     | Firecracker MicroVMs (e.g., Vercel Sandbox)                              |
|------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Isolation Mechanism    | Software-based; relies on namespaces and cgroups.                                     | Hardware-based; full virtualization boundary.                            |
| Kernel Architecture    | Shares the host OS kernel across all containers.                                      | Dedicated, lightweight Linux kernel per sandbox.                         |
| Security Posture       | Suitable for trusted, first-party code; kernel escapes remain a viable threat vector. | Designed explicitly for untrusted, adversarial code execution.           |
| Startup Latency        | Sub-second.                                                                           | Milliseconds; highly optimized for real-time agent latency.              |
| Primary Agent Use Case | Packaging and deploying final applications.                                           | Running arbitrary AI-generated scripts safely during the reasoning loop. |

While MicroVMs successfully prevent an adversarial agent from destroying the host operating system, they represent a perimeter defense that is ultimately incomplete. If an agent is granted an API key or network access to perform a legitimate task, a MicroVM cannot prevent the agent from abusing those authorized credentials to delete a production database or exfiltrate sensitive data to an external server.<sup>42</sup> True security for autonomous agents requires shifting from broad perimeter defense to granular, capability-based security architectures.

## 5. Mitigating RCE: WASI, MCP, and Capability-Based Security

If MicroVMs provide robust isolation, capability-based security provides granular, unbypassable control over intent and authorization. In a true capability-based system, an application or agent possesses zero implicit rights upon execution. It cannot read the filesystem, open a network socket, or access system time unless it is explicitly handed a "capability token" or a handle that

grants permission for that specific, narrowly scoped action.<sup>14</sup>

## 5.1 The WebAssembly System Interface (WASI) and Linear Memory

WebAssembly (Wasm) and the WebAssembly System Interface (WASI) have emerged as the premier execution substrates for capability-based agent runtimes.<sup>43</sup> Wasm operates on a shared-nothing "Linear Memory" model. When a Wasm module—representing the compiled logic of the agent—is executed, it is allocated a single, contiguous block of raw bytes.<sup>45</sup> The runtime (such as Wasmtime or Wasmer) enforces strict bounds checking, verifying that every memory load or store instruction remains within the 0 to `max_memory` range.<sup>45</sup> The module has absolutely no ability to address the memory of the host process.

Furthermore, WASI implements capability-based security for system interactions. An agent cannot simply invoke a standard C library function like `fopen("/etc/passwd")`. Through mechanisms like `libpreopen`, the agent must be explicitly injected with file descriptors representing the specific directories it is authorized to access at startup.<sup>14</sup> This transparent implementation ensures that malicious code cannot traverse outside its prescribed operational boundary.

## 5.2 CapSeal: Extending Capabilities to the Application Layer

This capability model is being formally extended to the unique operational characteristics of AI agents via academic frameworks such as CapSeal (Capability-Sealed Secret Mediation). CapSeal reframes secret handling for agentic systems, moving away from the dangerous paradigm of "hand the model a static API key" to "grant the model a narrowly scoped, non-exportable action capability".<sup>15</sup>

CapSeal binds agent sessions utilizing anti-replay mechanisms and implements defense-in-depth through multi-layered constraints.<sup>15</sup> Most notably, Layer 3 of the CapSeal architecture automatically derives lightweight defensive constraints directly from tool input schemas.<sup>46</sup> For example, if an agent is granted a capability to write a note, the schema constraint will identify parameters with names suggesting free-text fields (e.g., `note`, `comment`, `description`) and automatically enforce strict length limits and credential-pattern checks.<sup>46</sup> Optional parameters, which represent the most common exfiltration vectors, are strictly bounded, and filesystem parameters named `path` or `directory` receive sensitive-path blocking by default.<sup>46</sup> This ensures that even if an agent is successfully prompt-injected, the capability token it holds cannot be abused to exfiltrate bulk data or access forbidden paths.

## 5.3 The Model Context Protocol (MCP) as the Universal Connector

The integration of these secure tools and capabilities is increasingly standardized through the Model Context Protocol (MCP).<sup>47</sup> Introduced by Anthropic, MCP provides a universal, open-source standard for connecting AI applications to external systems via a structured JSON-RPC 2.0 interface.<sup>47</sup>

MCP defines three core connection primitives:

1. **Resources:** Exposing file-like data that an agent can read for context (e.g., API



responses, local file contents).<sup>47</sup>

2. **Tools:** Functions that the language model can invoke to perform an action.<sup>47</sup>

3. **Prompts:** Predefined prompt templates to accomplish specific tasks.<sup>50</sup>

By marrying the Model Context Protocol with the Wasm Component Model, agents can import and export specific MCP tool functions as strictly defined, language-agnostic capabilities.<sup>44</sup>

This eliminates boilerplate integration code and ensures that all external interactions are mediated through a unified, schema-validated protocol, dramatically reducing the attack surface.

## 5.4 Hardware-Enforced Capabilities: The Promise of CHERI

While software-based WASI capabilities and MCP gateways provide robust defense, they are ultimately reliant on the integrity of the underlying Just-In-Time (JIT) compiler. If the compiler contains logic bugs, adversarial code can elide bounds checks and pierce the linear memory sandbox.<sup>45</sup>

To achieve absolute containment, architectural research is migrating toward hardware-enforced capabilities, most notably CHERI (Capability Hardware Enhanced RISC Instructions).<sup>52</sup> CHERI extends conventional Instruction-Set Architectures (ISAs) to enable fine-grained memory protection and software compartmentalization directly at the silicon level.<sup>53</sup> It introduces architectural capabilities—unforgeable tokens of authority—that replace traditional raw memory pointers. Even if an AI agent generates a sophisticated buffer overflow exploit that bypasses the software compiler's checks, the CHERI hardware will immediately trap the invalid memory access and halt execution, physically preventing arbitrary memory reads or control-flow hijacking.<sup>53</sup>

## 6. Compiler Architecture Trade-offs: The Case Against LLVM for Agent Runtimes

To execute agent-generated logic rapidly across diverse, modern hardware (ARM64, RISC-V, AMD64) within these capability-secured sandboxes, the programming language must rely on a highly efficient compiler backend. Traditional Ahead-of-Time (AOT) compilation introduces prohibitive latency; waiting seconds or minutes for code to compile destroys the iterative, millisecond-latency read-eval-print loop required by an agent debugging its own code in real-time.<sup>13</sup> Therefore, a Just-In-Time (JIT) compiler is structurally necessary. When building a high-assurance, Rust-based toolchain, the two primary candidates are the industry-standard LLVM and the specialized Cranelift framework.<sup>12</sup>

### 6.1 The Heavyweight Standard: LLVM

LLVM is the ubiquitous compiler backend utilized by major languages including Rust (rustc), C/C++ (Clang), and Swift.<sup>56</sup> It is renowned for its aggressive optimization passes and its ability to generate highly performant machine code, boasting extensive support for Single Instruction, Multiple Data (SIMD) operations which are critical for heavy computational workloads.<sup>12</sup>

However, when applied to the specific domain of compiling adversarial, agent-generated code, LLVM presents severe structural drawbacks:

1. **Prohibitive Compilation Latency:** LLVM is architected primarily for AOT optimization. Its initialization phases and complex optimization passes are extraordinarily slow compared to lightweight alternatives.<sup>57</sup> For small code snippets—which typify agentic tool generation—the initialization overhead can dominate the entire process, introducing unacceptable latency into the agent's feedback loop.<sup>13</sup>
2. **Codebase Complexity and Vulnerability:** The LLVM codebase is massive, encompassing over 20 million lines of complex C++ code.<sup>59</sup> It was not historically designed with the isolation of malicious input as a primary constraint. Feeding adversarial, agent-generated Intermediate Representation (IR) into LLVM exposes the host environment to complex compiler-crashing logic bugs and miscompilations; historical audits have noted hundreds of open miscompilation bugs within the LLVM tracker.<sup>59</sup>
3. **Reliance on Undefined Behavior:** Perhaps most critically, LLVM relies heavily on optimizing around Undefined Behavior (UB). The compiler assumes the input code is well-formed and uses UB to aggressively strip away logic it deems impossible. This approach is fundamentally counterproductive and highly dangerous in a sandbox environment where executing untrusted code requires absolute predictable determinism and failsafe behavior.<sup>55</sup>

# Compiler Backend Trade-offs for Agentic Runtimes

| Feature Domain            | LLVM             | Cranelift                     |
|---------------------------|------------------|-------------------------------|
| Implementation Language   | C++              | Rust                          |
| Lines of Code             | ~2 Million       | ~200,000                      |
| Compilation Speed         | Slow / AOT Focus | Fast / JIT Focus              |
| Execution Performance     | Highest          | ~14% slower than LLVM         |
| Undefined Behavior in IR  | Yes              | No                            |
| Malicious Input Hardening | Low              | High / Linear Time Algorithms |

While LLVM provides superior peak execution performance and mature SIMD support, Cranelift's lightweight Rust implementation, rapid compilation speed, and strict avoidance of Undefined Behavior make it the superior choice for JIT-compiling untrusted, agent-generated logic.

Data sources: [Cranelift.dev](#), [Stanford SFI Slides](#), [Umbra Compiler Study \(TUM\)](#), [Hacker News Discourse](#)

## 7. The Cranelift Alternative: High-Assurance JIT for Malicious Inputs

To circumvent the inherent risks of LLVM, the ideal compiler backend for an agent-oriented language must be designed from the ground up for security and rapid compilation. Cranelift, a project developed by the Bytecode Alliance, fulfills these precise criteria.<sup>55</sup> Cranelift is a relatively simple, innovative compiler backend written entirely in Rust, guaranteeing memory safety in the compiler itself and eliminating entire classes of buffer overflow vulnerabilities that plague C++ compilers.<sup>55</sup> It serves as the primary JIT engine for the Wasmtime WebAssembly virtual machine, successfully executing sandboxed, untrusted code in production environments globally.<sup>55</sup>

Cranelift represents the optimal architectural choice for an agent programming language for several critical reasons:

1. **Compilation Speed over Peak Optimization:** Cranelift is developed with the speed of the compiler itself as a primary metric. It has been measured to perform code-generation approximately an order of magnitude faster than an equivalent LLVM-based system, achieving compilation latencies under 200 milliseconds.<sup>13</sup> While the resulting execution may run approximately 14% slower than heavily optimized LLVM output<sup>55</sup>, this is a highly favorable trade-off for short-lived, IO-bound agent tool executions where startup time is paramount.
2. **Eradication of Undefined Behavior:** Unlike LLVM, Cranelift's Intermediate Representation (IR) is meticulously designed without undefined behavior.<sup>55</sup> This design choice eliminates subtle miscompilations, ensuring that the code generated by an agent executes exactly as specified in the IR, a requirement for deterministic auditing.
3. **Explicit Hardening Against Malicious Input:** Cranelift is specifically hardened against adversarial compiler inputs, mimicking the defensive posture of browser JIT engines.<sup>55</sup> It guards against pathological compilation runtimes by utilizing input-length-bounded recursion and strictly avoiding algorithms with quadratic or higher time complexities, effectively neutralizing compiler-targeted Denial of Service (DoS) attacks.<sup>55</sup> This focus on security is empirically validated; since its inception, Cranelift has experienced only two sandbox-escape-capable miscompilation CVEs.<sup>59</sup>
4. **Verification and the ISLE DSL:** The Cranelift development team utilizes advanced verification techniques, including fuzzing and symbolic translation validation, to ensure the register allocator and lowering rules maintain semantic correctness.<sup>55</sup> Furthermore, Cranelift utilizes a custom domain-specific language called ISLE for optimization rewrite rules, allowing for high-level integration with formal verification tools.<sup>55</sup>
5. **Multi-Architecture Hardware Targeting:** Cranelift is highly retargetable and actively compiles down to major modern architectures, including x86\_64, AArch64 (ARM64), and critically, RISC-V 64-bit.<sup>55</sup> The recent elevation of the 64-bit RISC-V Linux port of Rust to a Tier-1 ("Guaranteed to work") target ensures enterprise-grade reliability and seamless integration for next-generation open-hardware platforms.<sup>63</sup>

## 8. Proposed Architecture: The "Aegis" Language and High-Security JIT Compiler

Based on the preceding analysis of Vercel Zero's agent-centric syntax, the necessity of capability-based security, and the architectural superiority of Cranelift for adversarial JIT compilation, this research proposes a comprehensive specification and implementation strategy for a novel agent-oriented language, codenamed **Aegis**.

Aegis is designed as a highly deterministic, capability-bound systems language. The Aegis compiler and runtime will be implemented entirely in Rust, utilizing Cranelift to provide a high-security, near-instantaneous JIT compilation pipeline targeting ARM64, RISC-V, and AMD64 hardware.

## 8.1 Language Specification and Syntax Design

Aegis shares the structural ethos and minimalistic philosophy of Zero, but it rigorously enforces formal verification and capability constraints directly at the syntax level.<sup>9</sup>

### Core Linguistic Principles:

- **Total Eradication of Global State:** Aegis code operates in absolute isolation. It cannot access any variables, APIs, or resources outside its immediate, explicitly defined scope.
- **Explicit Context and Capability Passing:** All side effects, standard output, filesystem interactions, and external tool executions must be passed into functions explicitly via an immutable Context object.<sup>9</sup> This mimics Zero's World object but extends it with generic type constraints representing capability tokens.<sup>66</sup>
- **Containment of Control Effects:** Aegis strictly eschews exceptions, which can leak information about the host environment through stack traces. Instead, agent functions must return deterministic Result types (e.g., Ok or Err), wrapping failure values securely to prevent side-channel data leakage.<sup>66</sup>
- **Deterministic Operations:** Floating-point math and concurrency models are tightly constrained to ensure cross-platform determinism, neutralizing a significant source of unpredictability in LLM operations.<sup>18</sup>
- **JSON-Native Feedback Loop:** Like Zero, Aegis outputs 100% of its diagnostic data, AST parsing errors, and type-check failures as highly structured, stable JSON payloads.<sup>9</sup> This removes the necessity for the LLM to parse human language to correct its code.

### Syntax Example (Aegis .ag source file):

Rust

```
// Aegis strictly enforces capability constraints in the function signature.
// The compiler statically verifies that the context holds these exact generic capabilities.
pub fun analyze_network_log(ctx: Context<Read<"/var/logs">, MCP<"NetworkSearch">>) ->
Result<i32, JSONError> {
    // If the agent attempts to read an unauthorized path, execution fails at compile time.
    let log_data = try ctx.fs.read_file("error.log");

    // The MCP capability is invoked. No raw HTTP requests are allowed.
    let search_result = ctx.mcp.call("NetworkSearch", log_data);

    return Ok(search_result.match_count());
}
```

## 8.2 The Rust-Based Compiler Frontend: The Capability Borrow Checker

The Aegis compiler frontend is implemented in safe Rust. Its primary responsibility is parsing the .ag source code into an Abstract Syntax Tree (AST), performing rigorous type and capability checking, and emitting Cranelift Intermediate Representation (CLIF).

1. **Lexical Analysis and Parsing:** The frontend tokenizes the input and constructs the AST. If the LLM generates a syntax error, the compiler generates a JSON payload detailing the exact line, character, and missing token, alongside a deterministic "diff" suggestion, enabling instantaneous automated repair.<sup>9</sup>
2. **Capability Resolution Pass:** Aegis introduces a novel compiler pass: capability resolution. Drawing inspiration from Rust's borrow checker—which tracks memory ownership—the Aegis compiler statically tracks capability consumption.<sup>66</sup> The compiler verifies that any function invoking an external tool has explicitly requested that capability in its signature. If an agent attempts to call a network API without the MCP<"NetworkSearch"> generic parameter, the compiler halts and emits a structured JSON capability error.<sup>15</sup>
3. **Lowering to Cranelift IR:** The verified AST is mapped directly to Cranelift instructions. The AST nodes are traversed, and simple arithmetic operations are translated using Cranelift's builder routines, such as `builder.ins().iconst(...)` and `builder.ins().iadd(...)`.<sup>67</sup> The absence of undefined behavior in the Aegis language specification translates seamlessly into Cranelift's safe, UB-free IR.<sup>55</sup>

## 8.3 The High-Security Runtime and JIT Engine

The Aegis runtime is the execution envelope for the agent. It leverages the wasmtime and cranelift crates to build a custom JIT engine optimized for low-latency, secure execution.<sup>55</sup>

**Software Fault Isolation (SFI):** Aegis enforces a linear memory model natively.<sup>45</sup> When the JIT compiler allocates memory for the generated machine code, it isolates the execution. Memory access instructions generated by the compiler are automatically masked or bounds-checked by Cranelift to ensure they cannot read or write outside the allocated linear heap, effectively preventing buffer over-reads from accessing host memory.<sup>38</sup>

**Multi-Architecture JIT Assembly:** To support deployment across massive computing clusters and edge devices, the runtime dynamically detects the host architecture. Using patterns akin to the `jit-asm-llvm` Rust crate, the system interfaces with Cranelift to emit highly optimized native machine code for the target environment<sup>17</sup>:

- **AArch64 (ARM64):** Fully supported for high-efficiency execution on mobile devices, Apple Silicon, and cloud infrastructure like AWS Graviton.<sup>16</sup>
- **RISC-V 64-bit (riscv64gc):** Utilizing Rust's Tier-1 support for RISC-V Linux targets, Aegis compiles natively for next-generation open-hardware platforms, providing future-proof architectural support.<sup>16</sup>
- **AMD64 (x86\_64):** Standard support for legacy and current enterprise cloud infrastructure.<sup>16</sup>

By entirely bypassing LLVM, the startup delay for compiling and executing an agent's custom script is reduced to microseconds.<sup>13</sup> This facilitates a real-time, interactive coding loop where

the agent can rapidly generate code, receive JSON compiler feedback, patch the AST, and execute the binary, achieving unprecedented autonomy.<sup>4</sup>

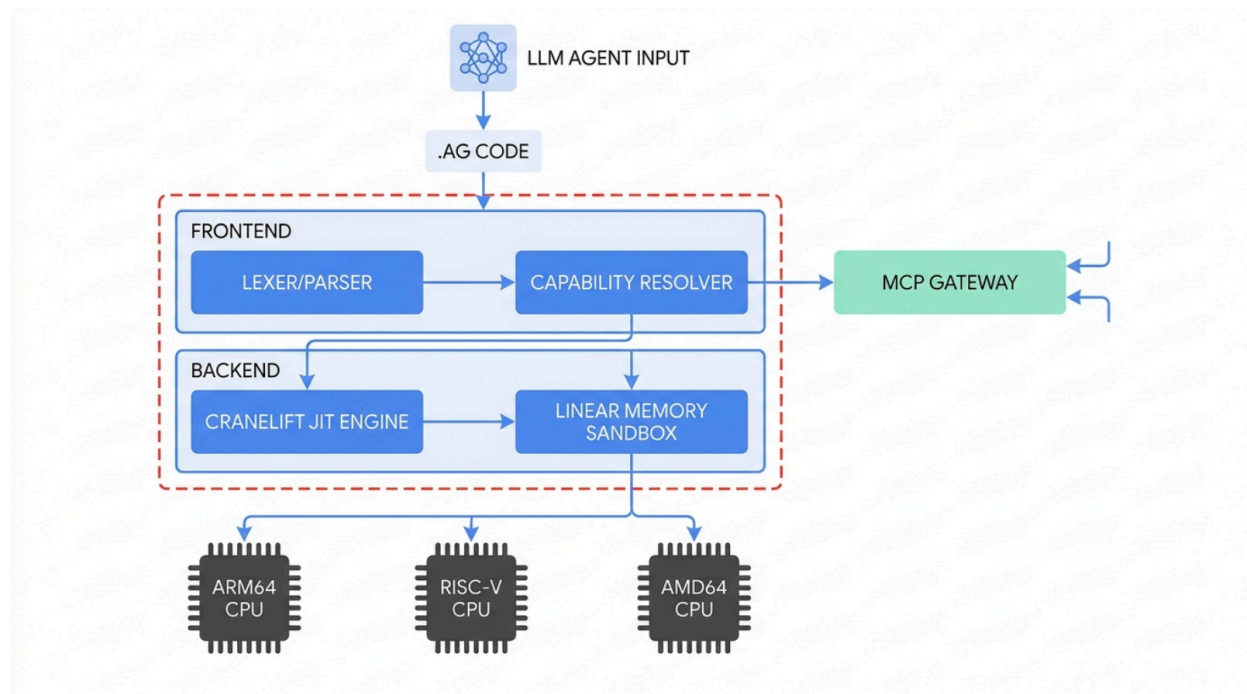
## 8.4 Native Model Context Protocol (MCP) Integration

Aegis embraces the Model Context Protocol as its universal standard library for external state and tool orchestration.<sup>68</sup> The Aegis runtime acts as an MCP Client.

When an agent writes code to interact with an external service (e.g., searching a database or querying a calendar), it does not write raw HTTP requests, utilize curl, or construct SQL strings. Instead, it invokes an MCP capability injected into its Context object. The Aegis runtime intercepts this function call within the Cranelift sandbox, strips it of any internal memory pointers, serializes the arguments into a secure JSON-RPC 2.0 payload, and forwards it to an external MCP server.<sup>49</sup>

This architecture enforces a strict structural boundary: the untrusted agent code running in the JIT sandbox can only communicate with the outside world through validated, schema-checked MCP bridges, effectively nullifying traditional network-based exfiltration attacks.<sup>46</sup>

## Aegis Runtime Architecture: Secure JIT Compilation Pipeline



The Aegis pipeline processes LLM-generated source code through a strict Rust-based capability checker before lowering it to Cranelift IR. Cranelift ensures rapid, secure JIT compilation to native ARM64, RISC-V, or AMD64 instructions, strictly confined within a Linear Memory sandbox and interfacing externally only via the Model Context Protocol (MCP).

## 9. Verification and Hardware Integration: The Path to Absolute Assurance

While the implementation of the Aegis compiler in memory-safe Rust using Cranelift secures the memory state and execution sandbox, it does not inherently guarantee the *logical* correctness of the agent's actions. Software Fault Isolation ensures the agent cannot crash the host, but it cannot guarantee the agent will correctly analyze a financial report or configure a network. To achieve total system assurance, the integration of formal methods into the compilation pipeline is necessary.<sup>64</sup>

Recent research indicates that bridging LLMs with formal verification tools, such as Dafny or Verus, can systematically eliminate hallucination in logical planning by enforcing mathematical rigor on the generated code.<sup>70</sup> Future iterations of the Aegis compiler architecture are specified to incorporate a theorem-proving pass. Before the AST is lowered to Cranelift IR, the compiler will generate mathematical proof obligations based on the capabilities requested and the logical branches present in the code.<sup>70</sup> Automated solvers, such as Z3, will attempt to discharge these obligations using Statistical Model Checking (SMC), mathematically verifying that the agent's code cannot violate defined security invariants or enter forbidden states during execution.<sup>70</sup>

Furthermore, as semiconductor hardware continues to evolve, the Aegis runtime is structurally positioned to adopt CHERI ISA extensions seamlessly.<sup>53</sup> Because Cranelift allows for flexible, retargetable code generation<sup>55</sup>, adding a backend target for a CHERI-enabled RISC-V architecture would allow Aegis to upgrade from Software Fault Isolation (SFI) to absolute, hardware-enforced pointer provenance.<sup>53</sup> This transition would secure the execution at the silicon level, providing unparalleled security guarantees without requiring any modifications to the agent's high-level source code.

## 10. Conclusion

The current trajectory of artificial intelligence development exposes a critical architectural vulnerability: the deployment of autonomous reasoning engines within execution environments designed for human developers. As demonstrated by the alarmingly high success rates of prompt-injection attacks, container escapes, and the proliferation of unverified shadow IT, utilizing standard programming languages and traditional containerization for autonomous AI is structurally unsound.<sup>4</sup>

Innovations such as Vercel Labs' "Zero" language establish the necessary syntactic paradigm shift—demanding explicit effects, eliminating implicit global state, and re-orienting compiler feedback from human-readable text to deterministic, structured JSON.<sup>9</sup> However, syntax alone cannot secure the execution substrate.

The "Aegis" architecture proposed in this research synthesizes these syntactic innovations with a rigorous, capability-based security model and a highly optimized, adversarial-resistant execution backend. By discarding the heavyweight, undefined-behavior-reliant LLVM



framework in favor of the Rust-native Cranelift JIT compiler<sup>13</sup>, Aegis achieves the microsecond compilation latencies required by iterative LLM reasoning loops, without sacrificing the memory safety and Software Fault Isolation necessary to contain adversarial code.<sup>38</sup>

Targeting Tier-1 hardware architectures including RISC-V, ARM64, and AMD64<sup>16</sup>, and strictly funneling external interactions through the standardized Model Context Protocol<sup>68</sup>, Aegis provides a comprehensive, impenetrable execution environment. This holistic approach bridges the fundamental gap between the stochastic, non-deterministic nature of large language models and the rigorous, verifiable guarantees required by modern enterprise software systems, establishing a secure foundation for the future of agent-oriented programming.

## Works cited

1. State of AI 2025: 100T Token LLM Usage Study - OpenRouter, accessed May 16, 2026, <https://openrouter.ai/state-of-ai>
2. The rise and potential of large language model based agents: a survey, accessed May 16, 2026, <http://scis.scichina.com/en/2025/121101.pdf>
3. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead - arXiv, accessed May 16, 2026, <https://arxiv.org/pdf/2404.04834>
4. Introducing the new v0 - Vercel, accessed May 16, 2026, <https://vercel.com/blog/introducing-the-new-v0>
5. AI Agents: What Would Be the Best Programming Language for LLMs? - AkitaOnRails.com, accessed May 16, 2026, <https://akitaonrails.com/en/2026/02/09/ai-agents-best-programming-language-for-llms/>
6. Non deterministic languages. How will future AIs communicate? | by swardley | MapAI, accessed May 16, 2026, <https://medium.com/mapai/non-deterministic-languages-2f635ecbdecd>
7. When prompts become shells: RCE vulnerabilities in AI agent frameworks - Microsoft, accessed May 16, 2026, <https://www.microsoft.com/en-us/security/blog/2026/05/07/prompts-become-shells-rce-vulnerabilities-ai-agent-frameworks/>
8. Formal Methods for Verification in Human-Agent Interaction - DiVA portal, accessed May 16, 2026, <https://www.diva-portal.org/smash/get/diva2:1950795/FULLTEXT01.pdf>
9. Zero: Vercel Labs' New Experimental Systems Language Built for AI Agents (Hello World: 16.2 KiB in 1ms), launched a few hours ago : r/compsci - Reddit, accessed May 16, 2026, [https://www.reddit.com/r/compsci/comments/1teym31/zero\\_vercel\\_labs\\_new\\_experimental\\_systems/](https://www.reddit.com/r/compsci/comments/1teym31/zero_vercel_labs_new_experimental_systems/)
10. Unveiling AI Agent Vulnerabilities Part II: Code Execution | Trend Micro (GB), accessed May 16, 2026, <https://www.trendmicro.com/vinfo/gb/security/news/cybercrime-and-digital-thre>

[ats/unveiling-ai-agent-vulnerabilities-code-execution](#)

11. The Dark Side of LLMs: Agent-based Attacks for Complete Computer Takeover - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2507.06850v4>
12. Performance Analysis of LLVM and Cranelift Codegen for Just-in-Time Compilation in Emulation, accessed May 16, 2026, <https://xkev.io/thesis.pdf>
13. Compile-Time Analysis of Compiler Frameworks for Query Compilation - Home-Pages an der School of Computation, Information and Technology, accessed May 16, 2026, <https://home.cit.tum.de/~engelke/pubs/2403-cgo.pdf>
14. WASI's Capability-based Security Model - Yuki Nakata, accessed May 16, 2026, <http://www.chikuwa.it/blog/2023/capability/>
15. CapSeal: Capability-Sealed Secret Mediation for Secure Agent Execution - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2604.16762v1>
16. jit\_assembler - Rust - Docs.rs, accessed May 16, 2026, <https://docs.rs/jit-assembler>
17. jit-assembler - crates.io: Rust Package Registry, accessed May 16, 2026, <https://crates.io/crates/jit-assembler>
18. Non-Deterministic Systems | Guild.ai, accessed May 16, 2026, <https://www.guild.ai/glossary/non-deterministic-systems>
19. The Randomness You Didn't Ask For: Understanding Non-Determinism in LLMs, accessed May 16, 2026, <https://www.arcticdba.se/posts/nondeterminism-part-1/>
20. Why do so many articles on llm adoption mention non-determinism as a main barrier?, accessed May 16, 2026, [https://www.reddit.com/r/LLMDevs/comments/1n4ax2j/why\\_do\\_so\\_many\\_articles\\_on\\_llm\\_adoption\\_mention/](https://www.reddit.com/r/LLMDevs/comments/1n4ax2j/why_do_so_many_articles_on_llm_adoption_mention/)
21. Efficiently Programming Large Language Models using SGLang - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2312.07104v1>
22. LMQL — SQL for Language Models - Medium, accessed May 16, 2026, <https://medium.com/data-science/lmql-sql-for-language-models-d7486d88c541>
23. GitHub - sgl-project/sglang: SGLang is a high-performance serving framework for large language models and multimodal models., accessed May 16, 2026, <https://github.com/sgl-project/sglang>
24. Anka: A Domain-Specific Language for Reliable LLM Code Generation - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2512.23214v1>
25. Intent Formalization: A Grand Challenge for Reliable Coding in the Age of AI Agents - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2603.17150v1>
26. Evaluating LLM-generated code for domain-specific languages: molecular dynamics with LAMMPS - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2603.20630v1>
27. A Detailed Comparison of Top 6 AI Agent Frameworks in 2026 - Turing, accessed May 16, 2026, <https://www.turing.com/resources/ai-agent-frameworks>
28. Architectural Design Decisions in AI Agent Harnesses - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2604.18071v1>
29. P1GPT: A Multi-Agent LLM Workflow Module for Multi-Modal Financial Information Analysis - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2510.23032v1>

30. Leveraging Reinforcement Learning for an Efficient Windows Registry Analysis during Cyber Incident Response - Preprints.org, accessed May 16, 2026, <https://www.preprints.org/manuscript/202501.0416/v2>
31. Automated UI Testing on Frontend of a Web Application - ResearchGate, accessed May 16, 2026, [https://www.researchgate.net/publication/381093196\\_Automated\\_UI\\_Testing\\_on\\_Frontend\\_of\\_a\\_Web\\_Application](https://www.researchgate.net/publication/381093196_Automated_UI_Testing_on_Frontend_of_a_Web_Application)
32. P1GPT: a multi-agent LLM workflow module for multi-modal financial information analysis - arXiv, accessed May 16, 2026, <https://arxiv.org/pdf/2510.23032>
33. How we made v0 an effective coding agent - Vercel, accessed May 16, 2026, <https://vercel.com/blog/how-we-made-v0-an-effective-coding-agent>
34. ZeroLang: When Vercel Built a Systems Programming Language for AI Agents - needhelp, accessed May 16, 2026, <https://needhelp.icu/blogs/zerolang-vercel-ai-agent-language>
35. GitHub - vercel-labs/zero: The programming language for agents, accessed May 16, 2026, <https://github.com/vercel-labs/zero>
36. Secure Boundaries: Understanding LLM Sandbox Environments - Sandgarden, accessed May 16, 2026, <https://www.sandgarden.com/learn/llm-sandbox>
37. Understanding Sandboxes - Vercel, accessed May 16, 2026, <https://vercel.com/docs/vercel-sandbox/concepts>
38. What Is an Agent Execution Sandbox? | Augment Code, accessed May 16, 2026, <https://www.augmentcode.com/guides/agent-execution-sandbox>
39. Vercel Sandbox is an ephemeral compute primitive designed to safely run untrusted or user-generated code. - GitHub, accessed May 16, 2026, <https://github.com/vercel/sandbox>
40. Vercel Sandbox, accessed May 16, 2026, <https://vercel.com/docs/vercel-sandbox>
41. Beyond Traditional Testing: Addressing the Challenges of Non-Deterministic Software, accessed May 16, 2026, <https://dev.to/aws/beyond-traditional-testing-addressing-the-challenges-of-non-deterministic-software-583a>
42. Establishing Runtime Security for Agentic AI - IBM, accessed May 16, 2026, <https://www.ibm.com/think/insights/agentic-ai-runtime-security>
43. MVVM: Deploy Your AI Agents—Securely, Efficiently, Everywhere - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2410.15894v2>
44. Building Complex Agentic Systems with WebAssembly - Tamal Dutta Chowdhury, accessed May 16, 2026, <https://tamal.tech/building-complex-agentic-systems-with-webassembly/>
45. The Wasm Breach: Escaping Backend WebAssembly Sandboxes | by InstaTunnel | Medium, accessed May 16, 2026, <https://medium.com/@instatunnel/the-wasm-breach-escaping-backend-webassembly-sandboxes-05ad426051fc>
46. Causality Laundering: Denial-Feedback Leakage in Tool-Calling LLM Agents - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2604.04035v1>
47. What is the Model Context Protocol (MCP)?, accessed May 16, 2026, <https://modelcontextprotocol.io/docs/getting-started/intro>

48. Model Context Protocol - Wikipedia, accessed May 16, 2026, [https://en.wikipedia.org/wiki/Model\\_Context\\_Protocol](https://en.wikipedia.org/wiki/Model_Context_Protocol)
49. gianlucamazza/orka: A modular agent orchestration ... - GitHub, accessed May 16, 2026, <https://github.com/gianlucamazza/orka>
50. Extend your agent with Model Context Protocol - Microsoft Copilot Studio, accessed May 16, 2026, <https://learn.microsoft.com/en-us/microsoft-copilot-studio/agent-extend-action-mcp>
51. Server-Side WebAssembly: Building portable and secure multi-language apps 1 - DOKUMEN.PUB, accessed May 16, 2026, <https://dokumen.pub/server-side-webassembly-building-portable-and-secure-multi-language-apps-1.html>
52. Black Hat Europe 2025 | Sponsors, accessed May 16, 2026, <https://blackhat.com/eu-25/event-sponsors.html>
53. dckc/awesome-ocap: Awesome Object Capabilities and Capability Security - GitHub, accessed May 16, 2026, <https://github.com/dckc/awesome-ocap>
54. Investigate WASM as a HAL executable format · Issue #2863 · iree-org/iree - GitHub, accessed May 16, 2026, <https://github.com/iree-org/iree/issues/2863>
55. Cranelift, accessed May 16, 2026, <https://cranelift.dev/>
56. Rust (programming language) - Wikipedia, accessed May 16, 2026, [https://en.wikipedia.org/wiki/Rust\\_\(programming\\_language\)](https://en.wikipedia.org/wiki/Rust_(programming_language))
57. MIR: A lightweight JIT compiler project | Red Hat Developer, accessed May 16, 2026, <https://developers.redhat.com/blog/2020/01/20/mir-a-lightweight-jit-compiler-project>
58. Ask HN: Recommendation for general purpose JIT compiler - Hacker News, accessed May 16, 2026, <https://news.ycombinator.com/item?id=31389024>
59. Lightweight Fault Isolation: Practical, Efficient, and Secure Software Sandboxing, accessed May 16, 2026, <https://www.scs.stanford.edu/~zyedidia/docs/slides/lfi-seclunch-2023-11-15.pdf>
60. Sandboxes? In my process? It's more likely than you think. - David Adrian, accessed May 16, 2026, <https://dadrian.io/blog/posts/memory-safety-and-sandboxes/>
61. Cross-Architecture Application Portability Using WebAssembly in Distributed Edge-Cloud Systems - ORBilu, accessed May 16, 2026, [https://orbilu.uni.lu/bitstream/10993/67902/1/Thesis\\_Orbilu.pdf](https://orbilu.uni.lu/bitstream/10993/67902/1/Thesis_Orbilu.pdf)
62. paiml/ruchyruchy: Ruchy that compile ruchy - GitHub, accessed May 16, 2026, <https://github.com/paiml/ruchyruchy>
63. Project RP004: Support for a 64-bit RISC-V Linux port of Rust to Tier-1, accessed May 16, 2026, <https://riseproject.dev/2025/04/15/project-rp004-support-for-a-64-bit-risc-v-linux-port-of-rust-to-tier-1/>
64. Agentic AI-based Coverage Closure for Formal Verification - arXiv, accessed May 16, 2026, <https://arxiv.org/html/2603.03147v2>
65. GitHub - Dicklesworthstone/pi\_agent\_rust: High-performance AI coding agent

- CLI written in Rust with zero unsafe code, accessed May 16, 2026,  
[https://github.com/Dicklesworthstone/pi\\_agent\\_rust](https://github.com/Dicklesworthstone/pi_agent_rust)
66. (PDF) Tracking Capabilities for Safer Agents - ResearchGate, accessed May 16, 2026,  
[https://www.researchgate.net/publication/401470828\\_Tracking\\_Capabilities\\_for\\_Safer\\_Agents](https://www.researchgate.net/publication/401470828_Tracking_Capabilities_for_Safer_Agents)
67. bytecodealliance/craneflight-jit-demo: JIT compiler and runtime for a toy language, using ... - GitHub, accessed May 16, 2026,  
<https://github.com/bytecodealliance/craneflight-jit-demo>
68. What is the Model Context Protocol (MCP)? - Databricks, accessed May 16, 2026,  
<https://www.databricks.com/blog/what-is-model-context-protocol>
69. Computing « Shahzad Bhatti, accessed May 16, 2026,  
<https://weblog.plexobject.com/archives/category/computing>
70. Agentic Verification of Software Systems - arXiv, accessed May 16, 2026,  
<https://arxiv.org/html/2511.17330v3>
71. ICML Poster Position: Trustworthy AI Agents Require the Integration of Large Language Models and Formal Methods, accessed May 16, 2026,  
<https://icml.cc/virtual/2025/poster/40101>
72. Towards an Agentic LLM-based Approach to Requirement Formalization from Unstructured Specifications - arXiv, accessed May 16, 2026,  
<https://arxiv.org/html/2604.18228v1>